Blockchain - NEP Sem VI

NMCPC62 : Cryptocurrency and Blockchain Development

Module - 4: Smart Contracts and Development

(7 Hours)

Instructors : Geocey Shejy, Lifna C S, Pradnya Raut

Department of Computer Engineering, VESIT, Mumbai

Topics to be covered

1. Introduction to Solidity Programming Language,
2. Setting Up Development Environments:
   - Truffle, Ganache, and Remix IDE,
3. Writing and Deploying Basic Smart Contracts,
4. Smart Contract Lifecycle,
5. Gas Optimization, and Fees
6. Advanced Solidity Concepts:
   - Libraries, Modifiers, and Events,
7. Debugging, Testing, and Security
8. Best Practices for Smart Contracts,
9. Integration of Smart Contracts with Blockchain Networks

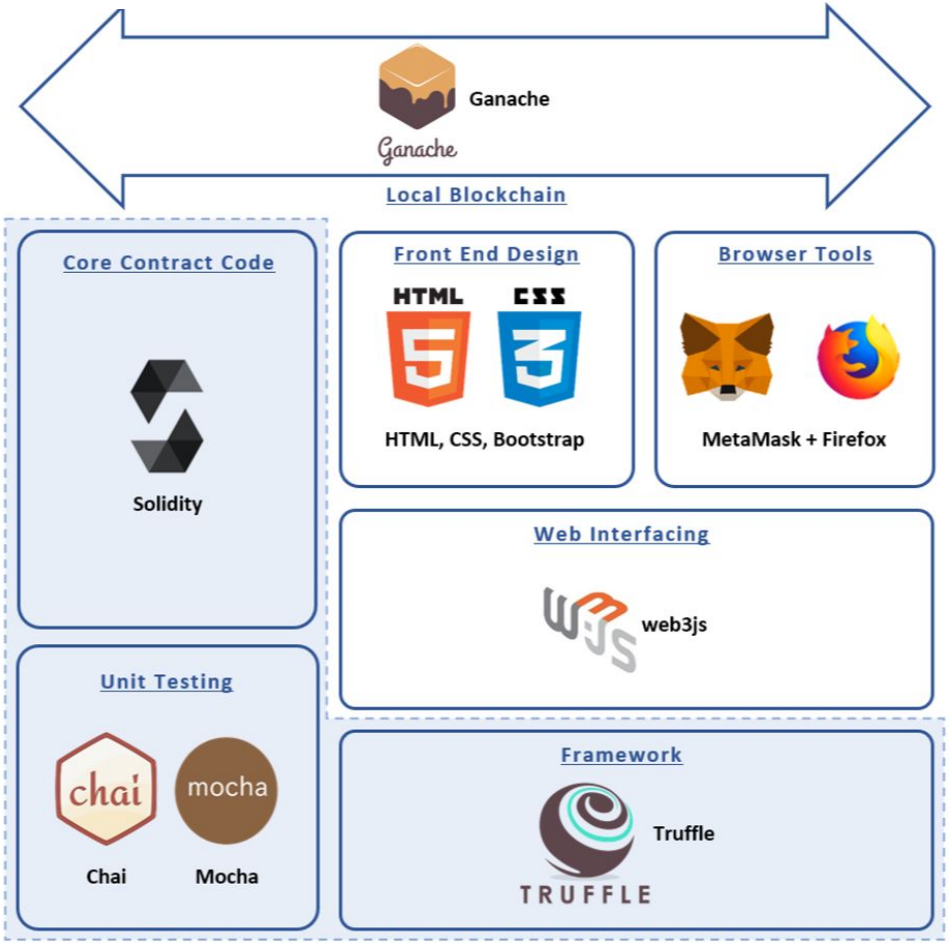
1. Introduction to Solidity Programming Language

- Refer the pdf attached Solidity Quick Guide here for Programming basics
- Remix IDE - LearnETH platform

1. Solidity Beginner Course (19 Assignments) - to understand the fundamentals of Solidity
2. Solidity ERC20 Token Course (6 Assignments) - Learn to understand and create ERC20 tokens.
3. Solidity NFT Course (6 Assignments) - Learn how to create your own NFTs (non-fungible tokens).
4. NFT Auction Contract (3 Assignments) - Learn how to write an ERC721 (NFT) auction contract.

2. Setting Up Development Environments

- Truffle
- Ganache
- Remix IDE - solidity Programming



Courtesy : Geeks4Geeks - Truffle Framework

2. Setting Up Development Environments: Truffle

Truffle :

- **development environment, testing framework and asset pipeline for blockchains using the Ethereum Virtual Machine (EVM)**, aiming to make life as a developer easier.

- Built-in smart contract compilation, linking, deployment and binary management.
- Advanced debugging with breakpoints, variable analysis, and step functionality.
- Use console.log in your smart contracts
- Deployments and transactions through MetaMask with Truffle Dashboard to protect your mnemonic.
- External script runner that executes scripts within a Truffle environment.
- Interactive console for direct contract communication.
- Automated contract testing for rapid development.
- Scriptable, extensible deployment & migrations framework.
- Network management for deploying to any number of public & private networks.
- Package management with NPM, using the ERC190 standard.
- Configurable build pipeline with support for tight integration.

Courtesy : Edureka

2. Setting Up Development Environments: Truffle

Benefits of Building with Truffle

1. **Insight into Transaction Details:** Truffle Dashboard lets you inspect transactions before they occur by giving you more digestible information to more confidently approve or reject transactions.
2. **Diagnose and fix errors quickly:** The Truffle Debugger together with Truffle's testing suite allows you to surgically diagnose errors when they occur and fix them quickly
3. **Focus on your dapp's uniqueness:** Truffle offers a workflow that gets out of your way and lets you focus on your dapp's unique functionality. It's prescriptive enough to provide guidance, yet flexible enough to adapt to your needs.
4. **Operate safely:** With Ganache's zero-config mainnet forking, you can easily test against live networks without spending real Ether. You can execute risky transactions in a sandbox and test integrations with production smart contracts, all with the same human-readable information and debugging tools you use for development networks.
5. **Applicable at all stages of development:** Regardless of whether your project began with Truffle CLI or another framework, Truffle can provide tooling and insights. We also support a number of different protocols and offer the ability to easily add support for new ones.

Courtesy : Truffle Suite

2. Setting Up Development Environments: Truffle

Truffle Suite Core Features and Tooling

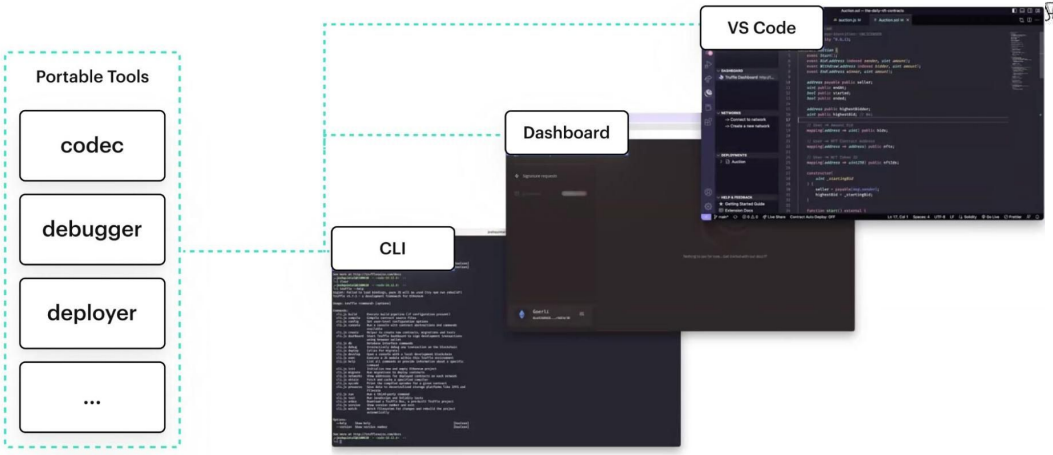
- a suite of tools that come together to create a smooth development experience.
- These tools include Truffle CLI, Truffle Dashboard, Ganache, and Truffle for VS code.
- Underlying libraries that power these tools.
    - **Truffle Debugger** (@truffle/debugger)
        - Portable Solidity debugger library that can be used with or without Truffle.
        - It is a standalone package that is available on NPM,
        - Truffle's debugger is at the heart of every Truffle tool to integrate it into Truffle Dashboard.
    - **Truffle Encoder & Decoder** (@truffle/codec)
        - provides an interface for decoding Solidity smart contract state and information sent to, or from smart contracts using the Solidity ABI.
        - It produces output in a machine-readable form that avoids losing any information.
        - This is a low-level package for encoding, decoding, and data representation.
    - **Truffle Fetch & Compile** (@truffle/fetch-and-compile)
        - used to obtain externally verified sources and compile them.

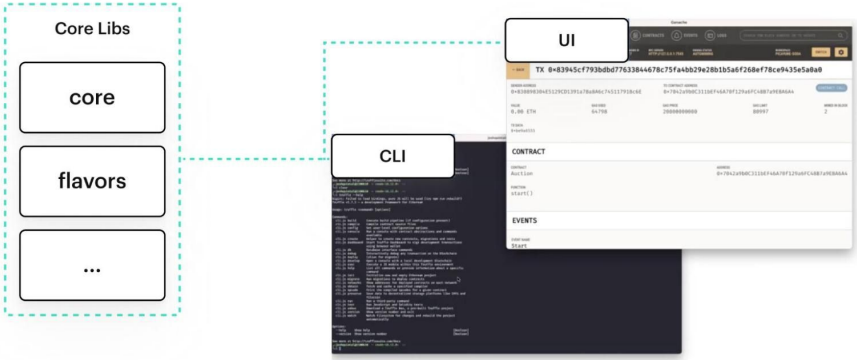
Courtesy : Truffle Suite

2. Setting Up Development Environments: Truffle

Truffle Suite Core Features and Tooling

Portable Libraries are combined to form various tools within Truffle Suite





Courtesy : Truffle Suite

2. Setting Up Development Environments: Truffle

Truffle Command Line Interface

- A powerful command-line tool that provides developers with a wide range of functionality for building, testing, and deploying smart contracts on the Ethereum blockchain.
- At its core, the Truffle CLI provides a suite of commands for compiling, testing, and deploying smart contracts, as well as for interacting with the Ethereum network.
- These commands can perform a wide range of tasks, from compiling contracts and generating boilerplate code to testing contracts and deploying them to the blockchain.
- One of the key features of the Truffle CLI is its integration with other Truffle tools and frameworks, such as
  - **Ganache UI (a local blockchain simulator)**,
  - **Drizzle (a front-end library for building dapps)**, and
  - **Web3.js (a library for interacting with the Ethereum network)**. This integration makes it easy for developers to build, test, and deploy complex dapps using a single, unified toolset.
- **Prerequisites:** Node.js (version 18 or later), NPM (Node Package Manager)

Courtesy : Truffle Suite

2. Setting Up Development Environments: Truffle

Truffle Basic Commands

1. **`truffle init`**: Initializes a new Truffle project in the current directory.
2. **`truffle compile`**: Compiles your Solidity smart contracts.
3. **`truffle migrate`**: Deploys your smart contracts to the blockchain.
4. **`truffle test`**: Runs unit tests for your smart contracts.
5. **`truffle console`**: Opens the Truffle console, allowing you to interact with your contracts using JavaScript.
6. **`truffle develop`**: Starts a development blockchain locally for testing and development.
7. **`truffle networks`**: Lists the available network configurations in your Truffle project.
8. **`truffle version`**: Displays the Truffle version installed.

Courtesy : Truffle Suite

2. Setting Up Development Environments: Truffle

Truffle - Smart Contract deployment on a Private Blockchain

1. Install Truffle Suite (Windows / Linux)
2. Create a Truffle Project / Unbox the existing Truffle project
3. Compile the Truffle Project
4. Test the Smart Contracts
5. Deploy the Smart Contract on Ganache
   a. Update the network settings in **truffle-config.js** file
   b. Create a Workspace in Ganache - append the truffle-config.js file
   c. Use **truffle migrate --reset** to compile and deploy the contracts on Ganache

(Github Repo - Truffle Tutorial)

Courtesy : Truffle Suite

2. Setting Up Development Environments: Ganache

Introduction to Ganache for Ethereum blockchain

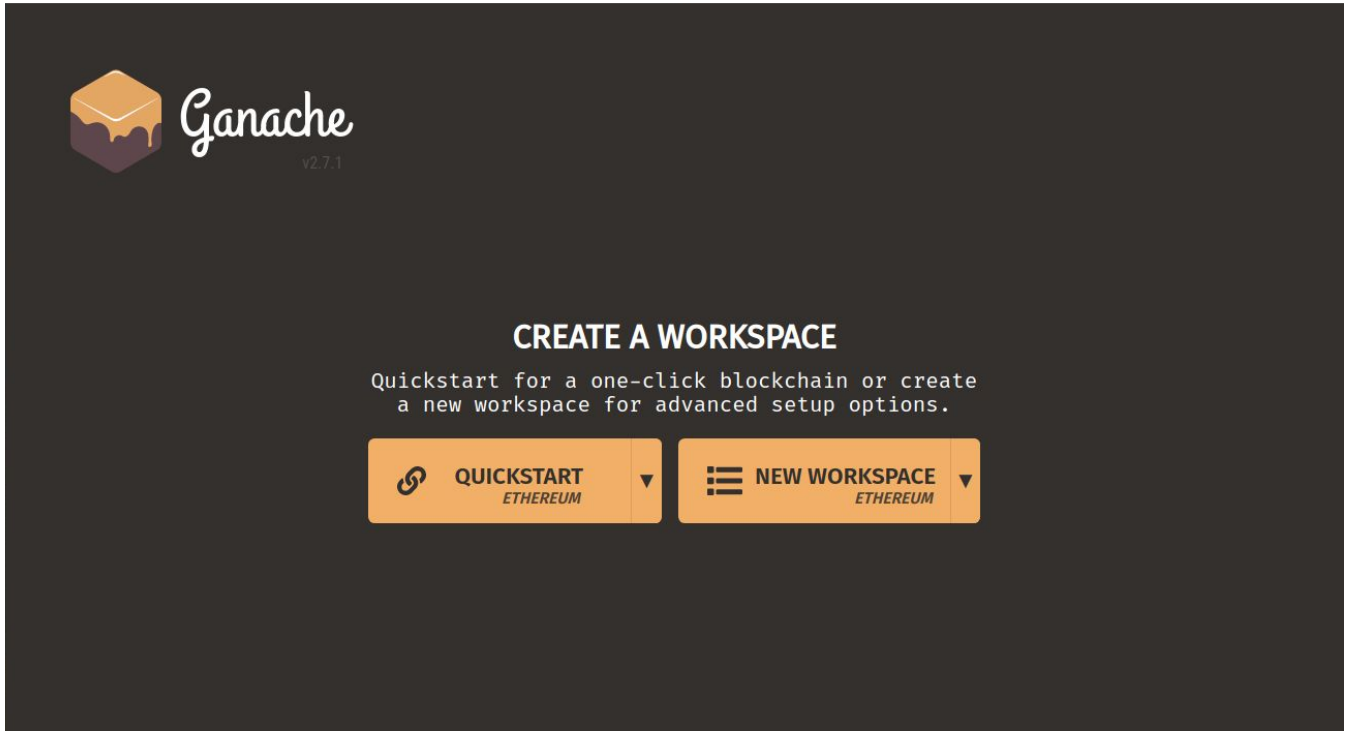
- Ganache is a personal blockchain for rapid Ethereum and Filecoin distributed application development.
- One can use Ganache across the entire development cycle;
- enabling you to develop, deploy, and test your dApps in a safe and deterministic environment.
- Ganache comes in two flavors: a UI and CLI.
  - Ganache UI is a desktop application supporting Ethereum and Filecoin technology.
  - More robust command-line tool, ganache, is available for Ethereum development. It offers:
    - console.log in Solidity
    - Zero-config Mainnet and testnet forking
    - Fork any Ethereum network without waiting to sync
    - Ethereum JSON-RPC support
    - Snapshot/revert state
    - Mine blocks instantly, on demand, or at an interval
    - Fast-forward time
    - Impersonate any account (no private keys required!)
    - Listens for JSON-RPC 2.0 requests over HTTP/WebSockets
    - Programmatic use in Node.js
    - Pending Transactions

All versions of Ganache are available for Windows, Mac, and Linux.

2. Setting Up Development Environments: Ganache

Introduction to Ganache for Ethereum blockchain

Flash screen of the Ganache after installation, Click on **Quickstart** to create a blockchain



Courtesy : Github - Ganache

2. Setting Up Development Environments: Ganache

Introduction to Ganache for Ethereum blockchain

On the Accounts tab, enlists the 10 Accounts created each with 100 Ethers

ACCOUNTS | BLOCKS | TRANSACTIONS | CONTRACTS | EVENTS | LOGS | SEARCH FOR BLOCK NUMBERS OR TX HASHES

CURRENT BLOCK	GAS PRICE	GAS LIMIT	HARDFORK	NETWORK ID	RPC SERVER	MINING STATUS	WORKSPACE		
0	20000000000	6721975	MERGE	5777	HTTP://127.0.0.1:7545	AUTOMINING	QUICKSTART	SAVE	SWITCH

MNEMONIC
outer tiger monitor doctor guess success select run globe fluid wish attend

HD PATH
m44'60'0'0account_index

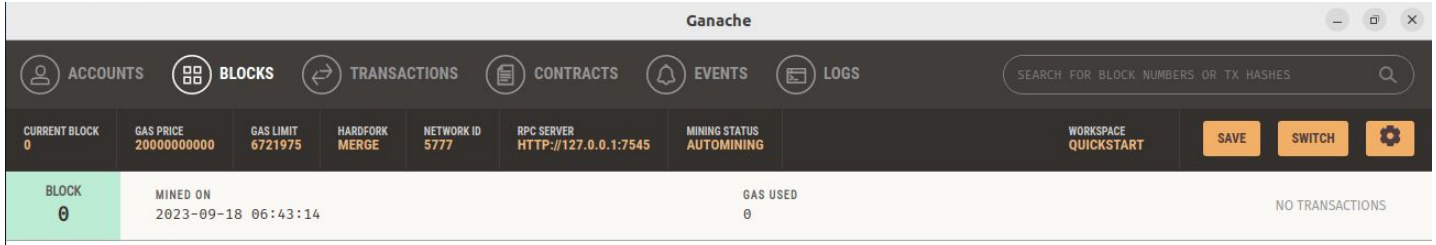
ADDRESS	BALANCE	TX COUNT	INDEX
0x3EF48Bde2FeD5515d6bA2bf7b12444F9fb05113f	100.00 ETH	0	0
0x59CB64541a112B554ED0beB8BB76Cac4A3264e7A	100.00 ETH	0	1
0xa2E518AC650a803D7d0Ca390287de4B85f8Ffa2C	100.00 ETH	0	2
0x7F191E74166dDab416877fEb83373BDe2484c857	100.00 ETH	0	3
0xB73E56fA400acD07221818109d1f87CC1741FDEE	100.00 ETH	0	4
0x2873707f7508406BefC0da18B11D597A8B8539C3	100.00 ETH	0	5

Courtesy : Github - Ganache

2. Setting Up Development Environments: Ganache

Introduction to Ganache for Ethereum blockchain

On the Blocks Tab, we could see the Genesis Block created

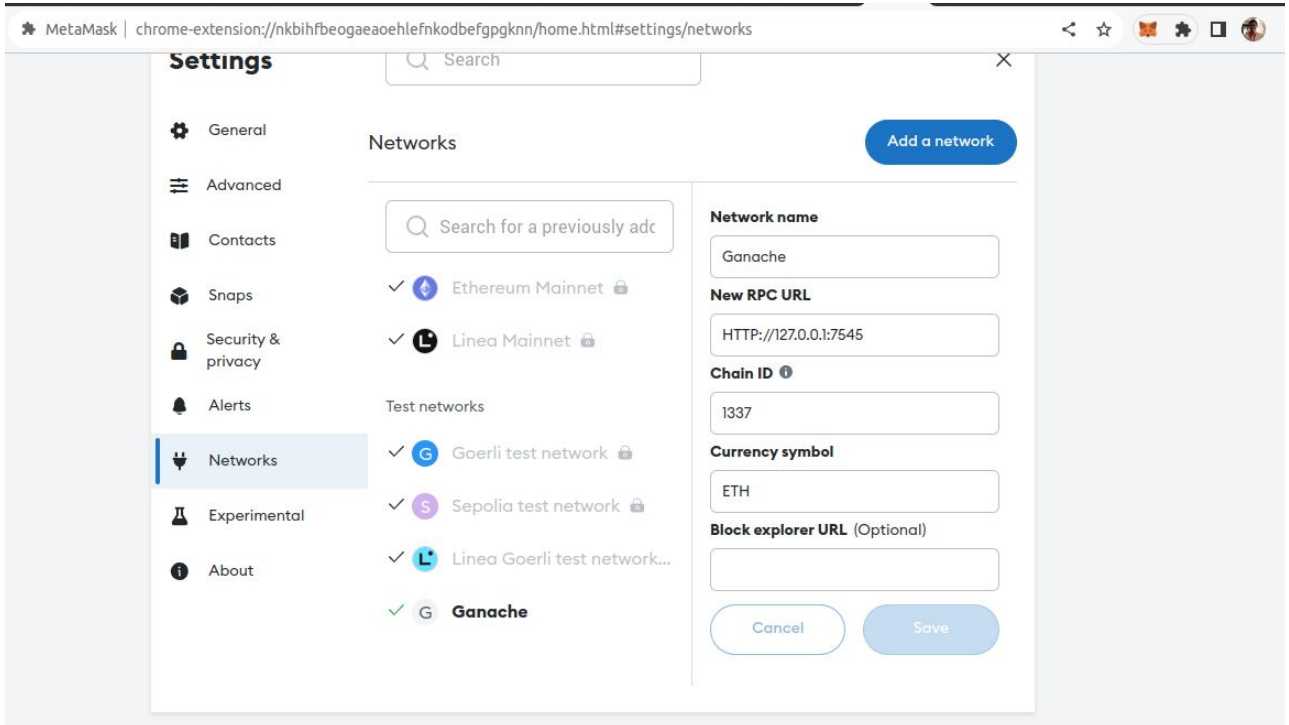


Courtesy : Github - Ganache

2. Setting Up Development Environments: Ganache

Introduction to Ganache for Ethereum blockchain

On the Metamask, add Ganache Network as follows:

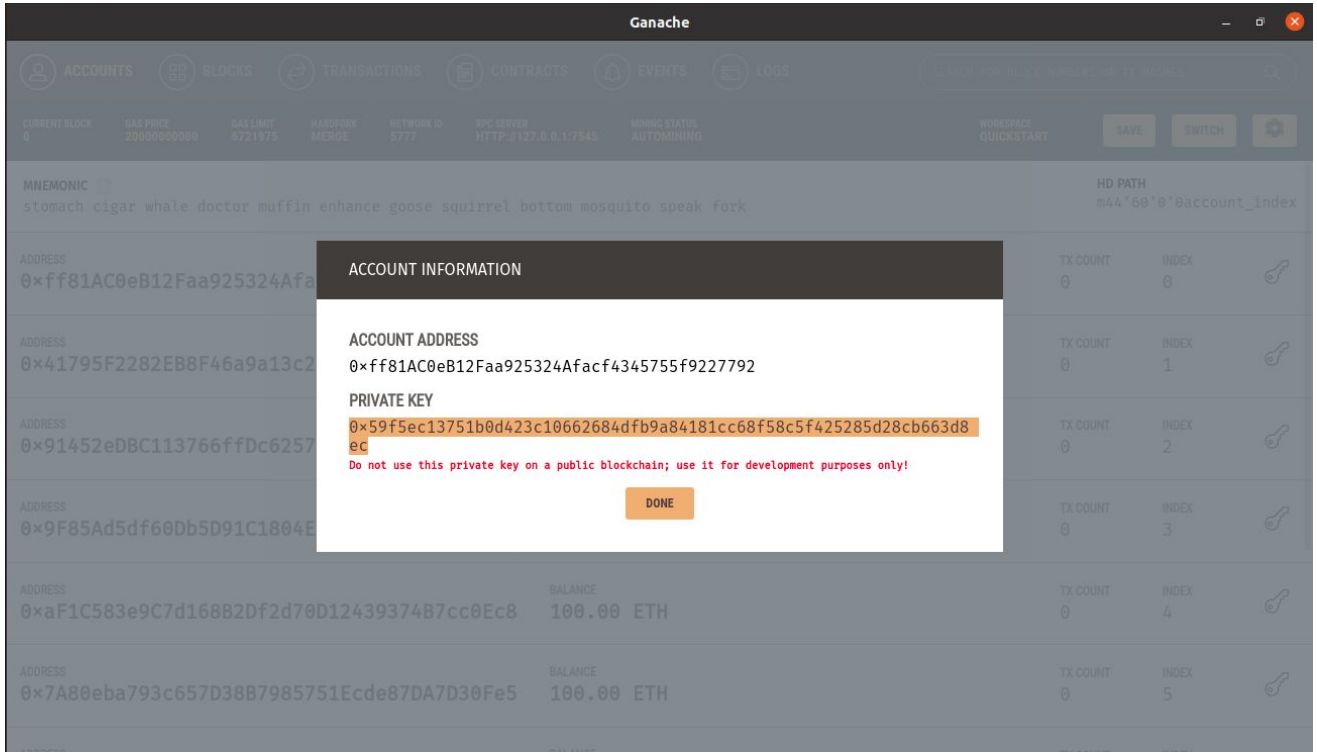


Courtesy : Github - Ganache

2. Setting Up Development Environments: Ganache

Introduction to Ganache for Ethereum blockchain

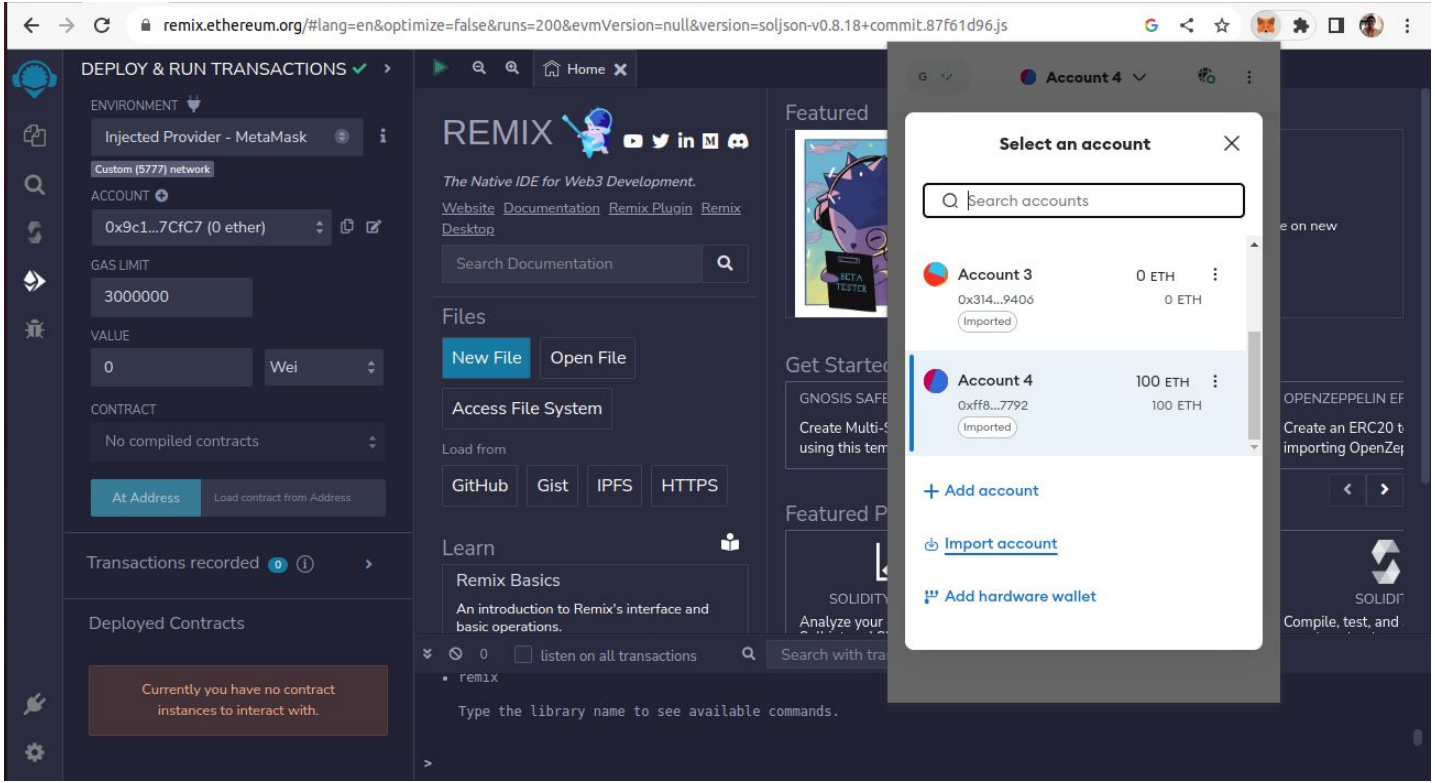
From on account, copy the Private key



Courtesy : Github - Ganache

Introduction to Ganache for Ethereum blockchain

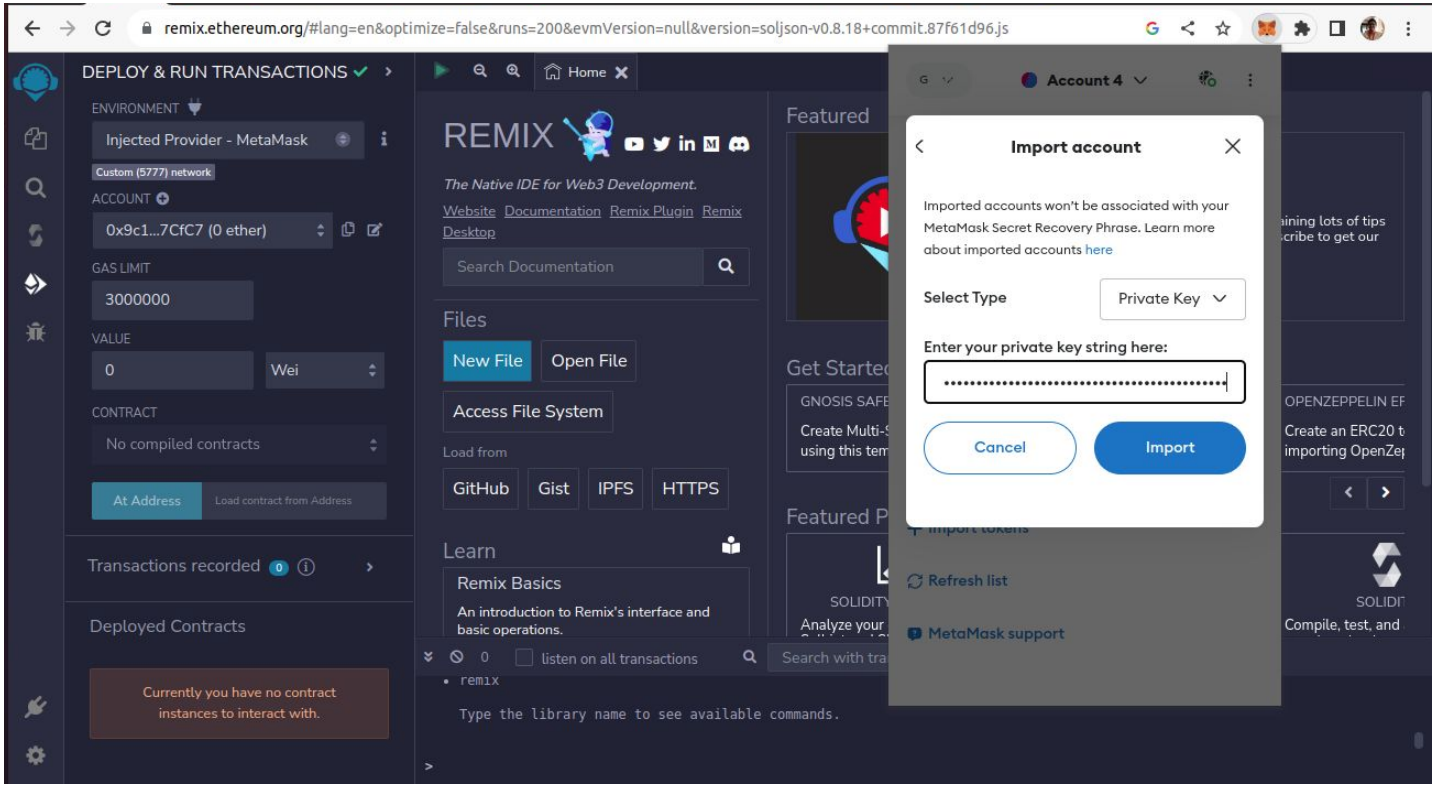
On the Metamask, select **Import Account**



Courtesy : Github - Ganache

Introduction to Ganache for Ethereum blockchain

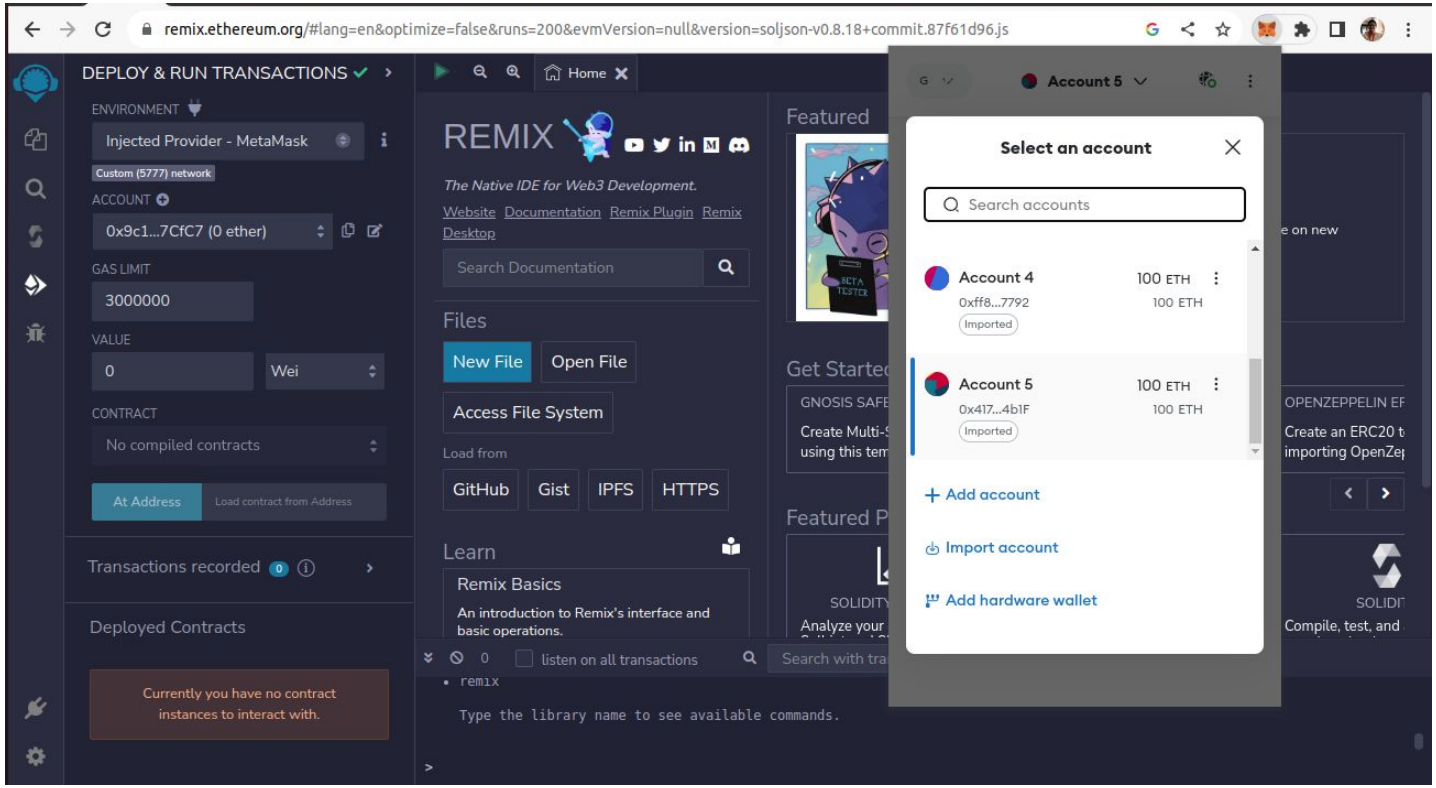
While importing Account on the Metamask, paste the Private key



Courtesy : Github - Ganache

Introduction to Ganache for Ethereum blockchain

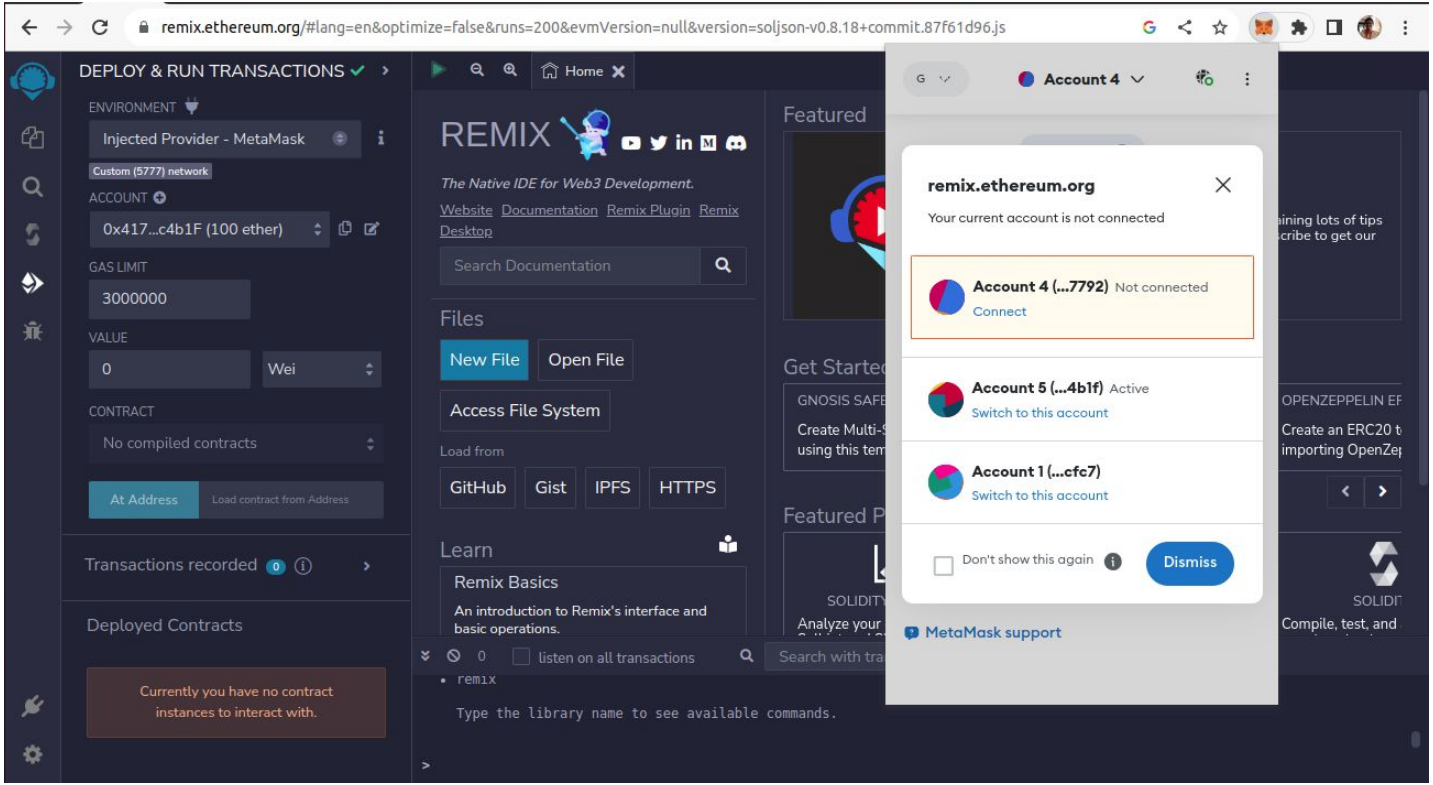
Enlists the Account imported



Courtesy : Github - Ganache

Introduction to Ganache for Ethereum blockchain

Select the Account from Metamask to be connected with Remix IDE



Courtesy : Github - Ganache

Introduction to Ganache for Ethereum blockchain

Deploy the Smart Contract

remix.ethereum.org/#lang=en&optimize=false&runs=200&evmVersion=null&version=soljson-v0.8.18+commit.87f61d96.js

DEPLOY & RUN TRANSACTIONS

ENVIRONMENT: Injected Provider - MetaMask
Custom (5777) network

ACCOUNT: 0xff8...27792 (100 ether)

GAS LIMIT: 3000000

VALUE: 0 Wei

CONTRACT: Storage - contracts/1_Storage.sol
evm version: paris

Deploy | Deploy - transact (not payable)

Publish to IPFS

At Address | Load contract from Address

Transactions recorded 1

Deployed Contracts

Home | 1_Storage.sol

```
// SPDX-License-Identifier: GPL-3.0
pragma solidity >=0.8.2 <0.9.0;
contract Storage {
    uint256 number;
    function store(uint256 num) public {    22520 gas
        number = num;
    }
    function retrieve() public view returns (uint256){
        return number;
    }
}
```

0 listen on all transactions | Search with transaction hash or addr

```
You can use this terminal to:
• Check transactions details and start debugging.
• Execute JavaScript scripts:
  - Input a script directly in the command line interface
  - Select a Javascript file in the file explorer and then run `remix.exe
command line interface
  - Right click on a JavaScript file in the file explorer and then click

The following libraries are accessible:
• web3 version 1.5.2
• ethers.js
• remix

Type the library name to see available commands.
creation of Storage pending...
>
```

Ganache

Account 4 → New contract

https://remix.ethereum.org

CONTRACT DEPLOYMENT

DETAILS | DATA

Site suggested

Gas (estimated) 0.00056555
0.00056555 ETH

Very likely in < 15 seconds Max fee: 0.00056555 ETH

Total 0.00056555
0.00056555 ETH

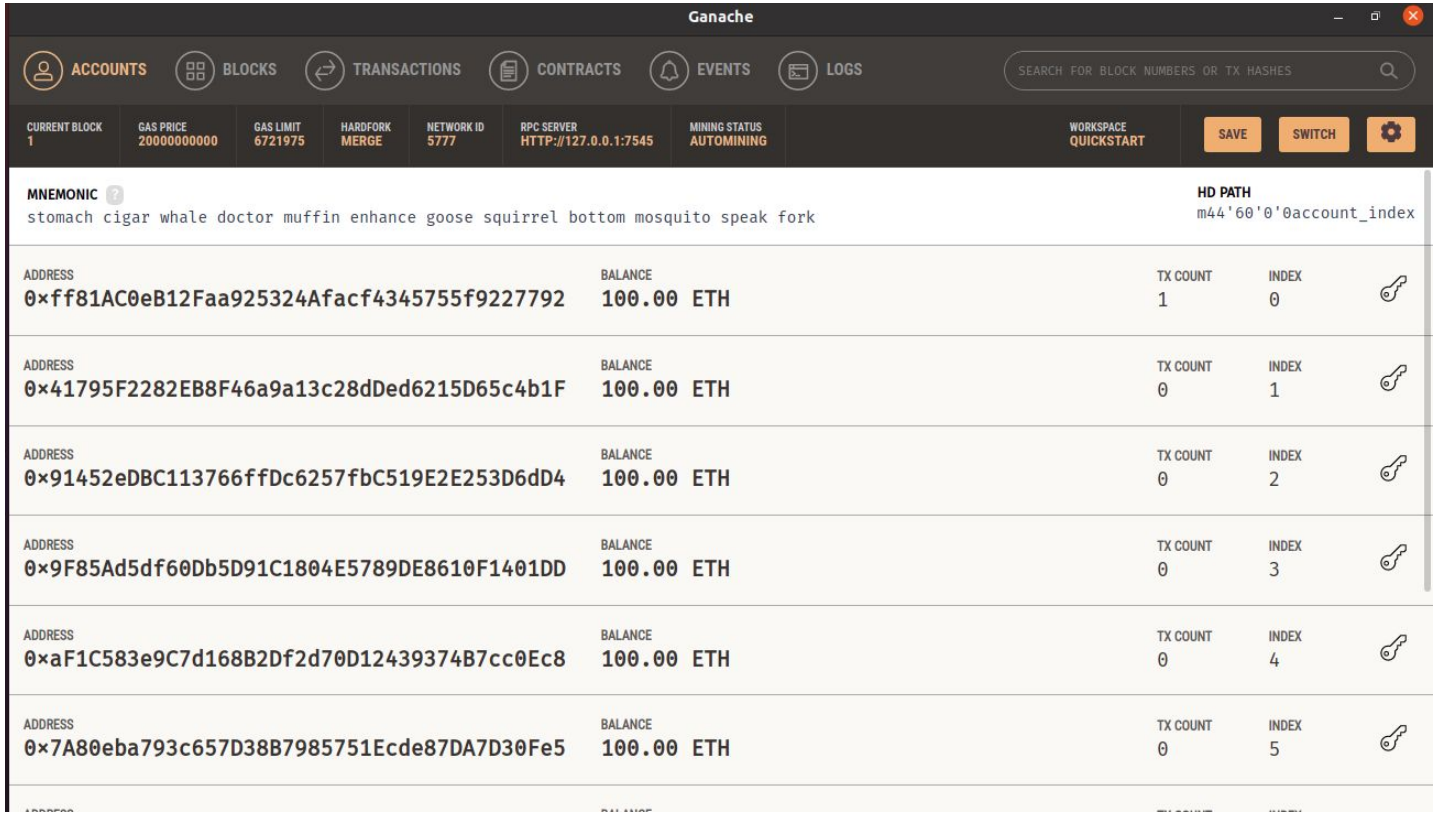
Amount + gas fee Max amount: 0.00056555 ETH

Reject | Confirm

Courtesy : Github - Ganache

Introduction to Ganache for Ethereum blockchain

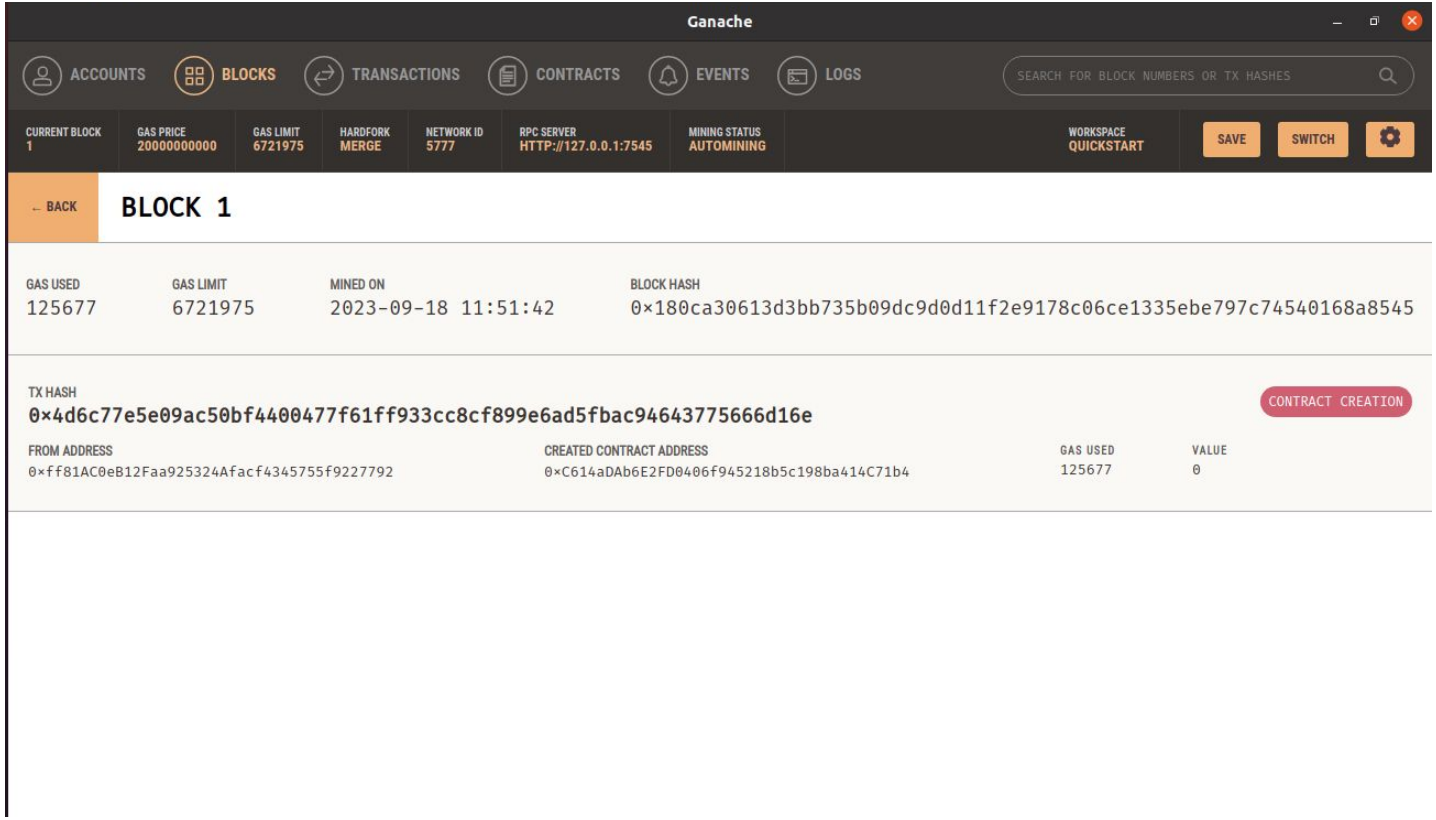
On the Ganache Environment, the Transaction count is updated



Courtesy : Github - Ganache

Introduction to Ganache for Ethereum blockchain

Block 1 is added to the Blockchain which displays the Contract Created



Courtesy : Github - Ganache

Introduction to Ganache for Ethereum blockchain

Transaction details of the Contract is displayed on Ganache Environment

Ganache

ACCOUNTS | BLOCKS | TRANSACTIONS | CONTRACTS | EVENTS | LOGS | SEARCH FOR BLOCK NUMBERS OR TX HASHES

CURRENT BLOCK	GAS PRICE	GAS LIMIT	HARDFORK	NETWORK ID	RPC SERVER	MINING STATUS	WORKSPACE		
1	20000000000	6721975	MERGE	5777	HTTP://127.0.0.1:7545	AUTOMINING	QUICKSTART	SAVE	SWITCH

← BACK

TX 0x4d6c77e5e09ac50bf4400477f61ff933cc8cf899e6ad5fbac94643775666d16e

SENDER ADDRESS
0xff81AC0eB12Faa925324Afacf4345755f9227792

CREATED CONTRACT ADDRESS
0xC614aDAb6E2FD0406f945218b5c198ba414C71b4

CONTRACT CREATION

VALUE	GAS USED	GAS PRICE	GAS LIMIT	MINED IN BLOCK
0.00 ETH	125677	3375000000	125677	1

TX DATA

0x6080604052348015610010576000080fd5b506101508061002060003960000f3fe6080604052348015610010576000080fd5b506004361061003576000356e01c80632e64cec11461003b
5780636057361d14610059575b600080fd5b610043610075565b604051610050919061006a1565b604051809103905f35b61007360048036038101906100 6e91906100ed565b61007e565b00
5b600080549050905 65b80600081905555050565b600081905091905056 5b61009b81610088565b82525050565b6000602082019050610066000830184610092565b92915050565b600080
fd5b6100ca81610088565b81146100d557600080fd5b50565b600081359050610100e7816100c1565b92915050565b600060208284031215610103576101026100bc565b5b60006101118482
85016100d8565b915050929150505 6fea2646970667358221220259 6eebd36c13936f52d856650b6b509a15fd4c949bcbcd94b3b50da3f29cce64736f6c63430008120033

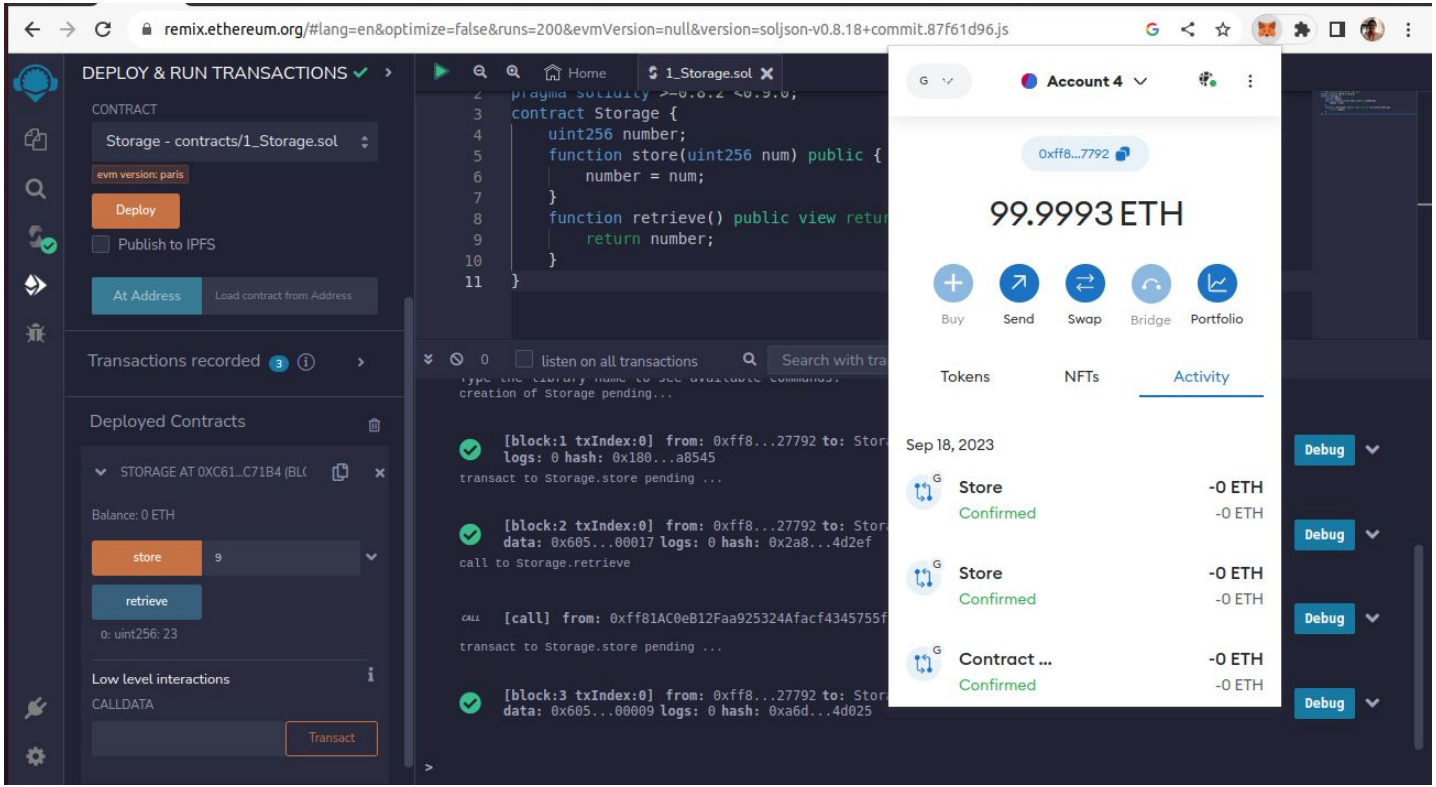
EVENTS

NO EVENTS

Courtesy : Github - Ganache

Introduction to Ganache for Ethereum blockchain

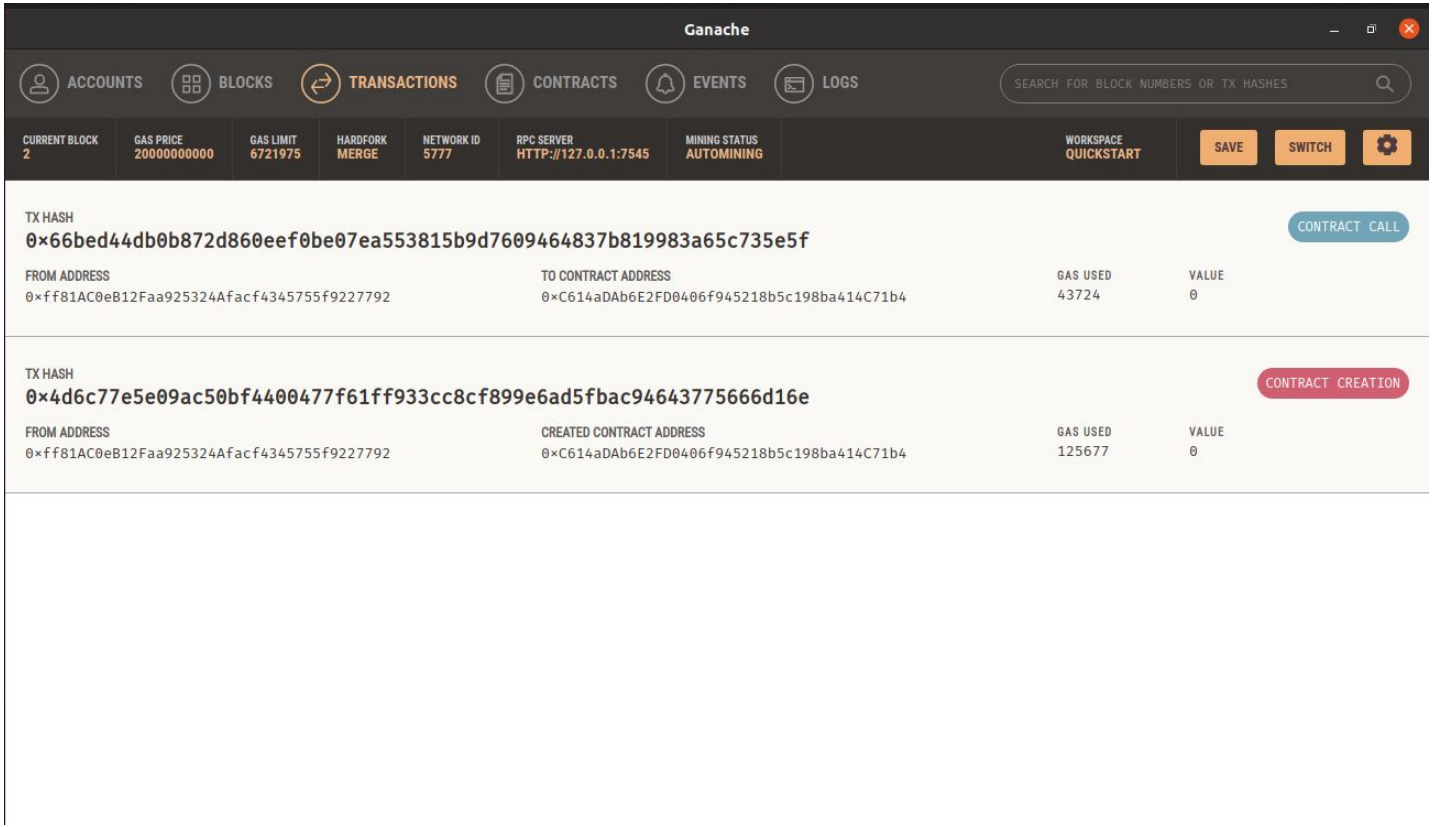
After interacting with the Smart Contract, funds are updated on the Metamask



Courtesy : Github - Ganache

Introduction to Ganache for Ethereum blockchain

On the Ganache, the Contract call is listed



Courtesy : Github - Ganache

2. Setting Up Development Environments: Remix IDE

- An open-source web-based IDE
- Designed for Solidity smart contract development
- No installation required (runs in browser)
- Provides built-in tools for:
  - Compilation
  - Deployment
  - Debugging
  - Testing
- Accessing Remix IDE : https://remix.ethereum.org/
  - No login required
  - GitHub integration is Optional

Advantages

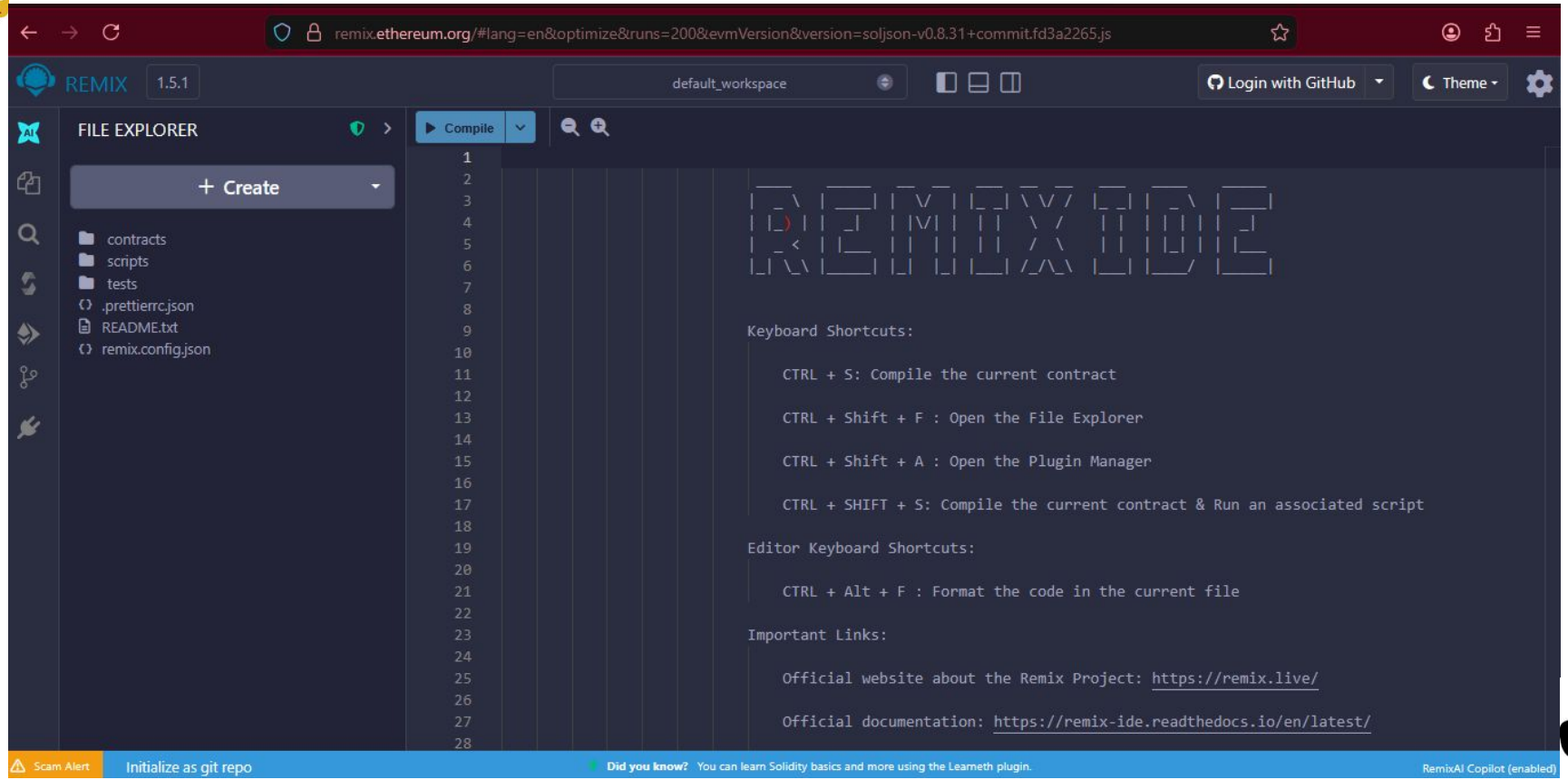
- No setup required
- Beginner-friendly
- Fast prototyping
- Built-in debugging tools

Limitations

- Not ideal for large-scale projects
- Limited version control compared to VS Code
- Less flexible than frameworks like Hardhat or Truffle Suite

REMIX IDE

2. Setting Up Development Environments: Remix IDE



REMIX IDE

Courtesy : Remix IDE

2. Setting Up Development Environments: Remix IDE

Remix has a modular plugin-based interface:

1. **File Explorer**

- Create and manage .sol files
- Default workspace: contracts/

2. **Solidity Editor**

- Write your smart contracts
- Syntax highlighting + auto-completion

3. **Compiler Module**

- Select Solidity version
- Compile contracts
- Shows errors and warnings

4. Deploy & Run Module

- Deploy contracts to:
  - JavaScript VM (local simulation)
  - Injected Provider (e.g., MetaMask)
  - Web3 Provider (testnets/mainnet)

5. Debugger

- Step-by-step execution of transactions
- View variables and stack

3. Writing and Deploying Basic Smart Contracts

Step 1: Create a New File

- Click File Explorer → New File
- Name it: SimpleStorage.sol

Step 2: Write Contract Code

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;
contract SimpleStorage {
   uint public data;
   function set(uint _data) public {
      data = _data;
   }
   function get() public view returns (uint) {
      return data;
   }
}
```

Step 3 : Compiling the Contract

- Go to **Solidity Compiler**
- Click **Compile SimpleStorage.sol**
- Ensure: No errors & Green check mark appears

Step 4 : Deploying the Contract

Using RemixVM Osaka (default)

- Go to **Deploy & Run Transactions**
- Click **Deploy**

Using MetaMask (Advanced)

- Install **MetaMask**
- Select Injected Provider
- Connect wallet
- Deploy to testnet (e.g., Sepolia)

3. Writing and Deploying Basic Smart Contracts

DEPLOY & RUN TRANSACTIONS

ENVIRONMENT — Reset State

Remix VM (Osaka)

VM

ACCOUNT

0x5B3...eddC4 (99.99999999999

+ Authorize Delegation

GAS LIMIT

Estimated Gas

Custom 3000000

VALUE

0 Wei

CONTRACT

SimpleStorage - contracts/1_Storage.s

evm version: osaka

Deploy

Compiled — 1_Storage.sol

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;
contract SimpleStorage {
    uint public data;
    function set(uint _data) public {    22492 gas
        data = _data;
    }
    function get() public view returns (uint) {    2431 gas
        return data;
    }
}
```

Explain contract

0 Listen on all transactions

Courtesy : Himanshi Blog

3. Writing and Deploying Basic Smart Contracts

DEPLOY & RUN TRANSACTIONS

Deploy

At Address | Load contract from Address

Transactions recorded 2

Deployed Contracts 2

STORAGE AT 0XD91...39138 (MEM

SIMPLESTORAGE AT 0XD8B...33F

Balance: 0 ETH

set | uint256 _data

data

get

Low level interactions

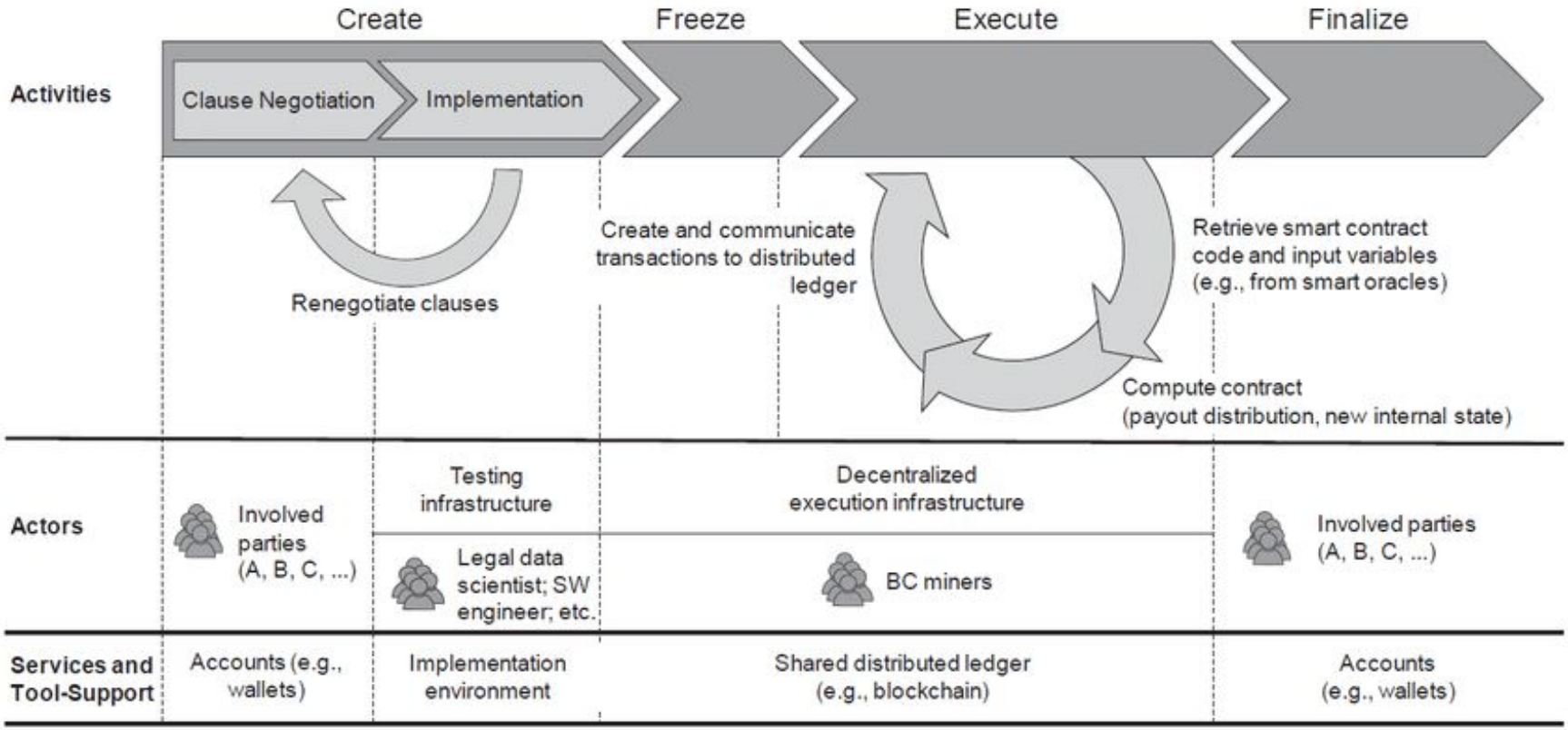
0 Listen on all transactions Filter with transaction hash or ad...

[vm] from: 0x5B3...eddC4 to: Storage.(constructor) value: 0 wei data: 0x608...f0033 logs: 0 hash: 0x6e4...d5367 Debug

status	1 Transaction mined and execution succeed
transaction hash	0x6e40da6f70552047c418eb560c4ad7cf08af669fc6bbf5cdf1c6ae5487ad5367
block hash	0x30b17f8413c8755e3449b28ce2a916b759ce58097eaa386581779c7d7aae3296
block number	1
contract address	0xd9145CCE52D386f254917e481eB44e9943F39138
from	0x5B38Da6a701c568545dCfcB03FcB875f56beddC4
to	Storage.(constructor)
transaction cost	117683 gas
execution cost	59509 gas
output	0x6080604052348015600e575f5ffd5b506004361060305 75f3560e01c80632e64cec1146034578063605736 1d14604e575b5f5ffd5b603a6066565b604051604591906 08d565b604051809103 90f35b6064600480360381019060609190 60cd565b606e565b005b5f5f54905090565b805f8190555050565b5f8190509190 5065b6087816077565b82525050565b5f602082019050609e5f8301846080565b929150 5065b5f5ffd5b60af816077565b811460b8575f5ffd5b50565b5f8135905060c78160a8565b92915050565b5f6020828403121560df5760de60a4565b5b5f60ea8482850160bb565b91505092915050 56fea2646970667358221220d534868d1b34f9d9a2fcac732259e55fd34242cbfa935062a0281d3fda07a95264736f6c634300081f0033
decoded input	{}
decoded output	-

REMIX IDE

4. Smart Contract Lifecycle



Courtesy : ResearchGate Paper

4. Smart Contract Lifecycle

1. **Creation Phase:**

- consists of iterative contract negotiation and an implementation phase.
- First, the parties must agree on the broad content and goals of the contract.
- Similar to typical contract negotiations and can be conducted online or in person contracts
- During this phase, the following tasks are completed:
    1. Multiple-party bargaining.
    2. Design, implementation, and validation of smart

2. **Freeze:**

- The validation of transactions on a blockchain is performed by a network of computers known as nodes. The blockchain miners are these nodes.
- To prevent the ecosystem from being swamped with smart contracts, miners must be paid a tiny fee in exchange for this service.

Courtesy : Medium

4. Smart Contract Lifecycle

3. **Execution:**

- Contracts placed on the distributed ledger are read by participating nodes.
- The authentication nodes validate the integrity of a smart contract,
- the code is performed by the smart contract interference engine (or by the compiler).
- When one party's inputs for execution are received in the form of coins (commitment to goods via coins), the interference engine **generates a transaction triggered by the met criterion**.
- The execution of the smart contract **results in a new set of transactions and a new state for the smart contract.**
- The **discoveries and new state information** are entered into the distributed ledger and validated using the consensus procedure.

4. **Finalize:**

- the resulting transactions and updated state information are recorded in the distributed ledger and confirmed via the consensus process.
- The previously pledged digital assets are transferred (assets are unfrozen), and the contract is signed to confirm all transactions.

Courtesy : Medium

5. Gas Optimization, and Fees

- Gas = unit that measures computational work required to execute operations
- Every operation (e.g., addition, storage, function call) consumes gas
- Paid in **Ether (ETH)**
- Example:
  - Simple transfer: ~21,000 gas
  - Smart contract execution: varies based on complexity
- **Why Gas Fees Fluctuate?**
  - Network congestion
  - Demand for transactions (e.g., NFTs, DeFi)
  - Block size limits
  - Note : During high demand, fees increase significantly

Gas Limit	Max gas user is willing to spend
Gas Price	Cost per unit gas (in Gwei)

Gas optimization

crucial for building efficient smart contracts.

Courtesy : Medium - Yamen M.

5. Gas Fee Structure

- After the EIP-1559 upgrade, gas fees consist of:

  1. Base Fee
     - Automatically adjusted by network
     - Burned (removed from circulation)
  2. Priority Fee (Tip)
     - Paid to miners/validators for faster inclusion

  Total Fee Formula :

$$\boxed{\text{Transaction Fee} = \text{Gas Used} \times (\text{Base Fee} + \text{Priority Fee})}$$

- Example Scenario :
  - Given Gas Used = 21,000, Base Fee = 30 Gwei, Priority Fee = 2 Gwei

    Total Fee:

$$\boxed{= 21,000 \times (30 + 2) = 672,000 \text{ Gwei} = 0.000672 \text{ ETH}}$$

5. Gas Optimization Techniques

1. **Storage Optimization**
   - Storage is the most expensive operation
   - Use: uint256 instead of smaller types (EVM optimized)
   - Packing variables in structs
   - Example :

```
struct Data {
   uint128 a;
   uint128 b; // packed efficiently
}
```

2. **Minimize Storage Writes**
   - Writing to blockchain = expensive
   - Use: memory instead of storage where possible
   - Avoid unnecessary state changes

3. Use view and pure Functions

- Do not cost gas when called externally
- Example

```
function get() public view returns(uint)
```

5. Gas Optimization Techniques

4. Avoid Loops Over Large Data

- Loops increase gas linearly

  for(uint i=0; i<users.length; i++)

- Use mappings
- Off-chain computation

5. Use Events Instead of Storage

- Logging via events is cheaper than storing data

  event DataStored(uint value);

6. Short-Circuit Conditions

- Put cheaper conditions first

  if (x > 0 && expensiveFunction())

7. Reuse Variables / Cache Values

- Avoid repeated storage reads

  uint temp = data;

5. Gas Optimization in Transactions

1. **Use Off-Peak Times**
   - Lower congestion = lower fees
2. **Set Optimal Gas Price**
   - Avoid overpaying
3. **Use Layer 2 Solutions**
   - Examples:
     - Polygon
     - Arbitrum
     - Optimism

Importance of Gas Optimization

- Reduces transaction costs
- Improves scalability
- Enhances user experience in DeFi & DApps
- Makes contracts more competitive

Tools for Gas Optimization

- Remix IDE – Gas estimation during deployment
- Hardhat – Gas reporter plugin
- Foundry – Advanced testing & gas profiling

Courtesy : Himanshi Blog

6.Advanced Solidity Concepts : Libraries

- Reusable pieces of code deployed once and used by multiple contracts
- Help reduce duplication and improve modularity
- Cannot hold state (no storage variables)
- Example :

```
library MathLib {
  function add(uint a, uint b) internal pure returns (uint) {
    return a + b;
  }
}
contract Test {
  function sum(uint x, uint y) public pure returns (uint) {
    return MathLib.add(x, y);
  }
}
```

Types of Library Functions

- internal → inlined into contract (gas efficient)
- external → deployed separately and called

Advantages

- Code reuse
- Gas optimization
- Better organization

6.Advanced Solidity Concepts : Modifiers

- Used to change behavior of functions
- Commonly used for access control and validation
- Example

```
contract Ownable {
  address public owner;
  constructor() {
    owner = msg.sender;
  }
  modifier onlyOwner() {
    require(msg.sender == owner, "Not owner");
    _;
  }
  function withdraw() public onlyOwner {
    // restricted logic
  }
}
```

Common Use Cases

- Access restriction (onlyOwner)
- Input validation
- Pausing contract execution

6.Advanced Solidity Concepts : Events

- Used for logging data on blockchain
- Stored in transaction logs (cheaper than storage)
- Useful for frontend interaction (DApps)
- Example

```
contract EventExample {
  event DataStored(address indexed user, uint value);

  function store(uint _value) public {
    emit DataStored(msg.sender, _value);
  }
}
```

Key Features

- indexed parameters → searchable in logs
- Accessible via Web3 libraries
-

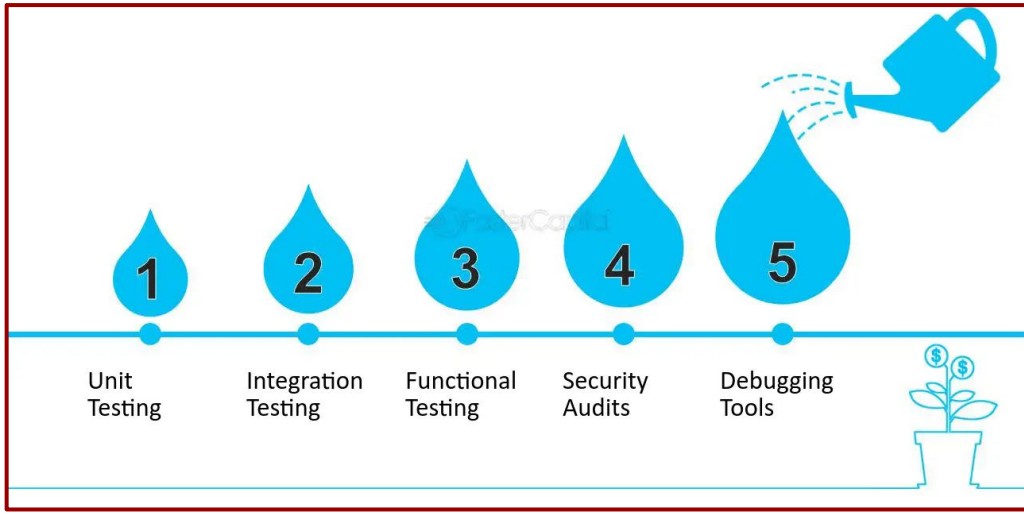
Benefits

- Gas efficient logging
- Enables real-time UI updates
- Transparency and traceability

7. Debugging, Testing, and Security

Common Debugging Techniques

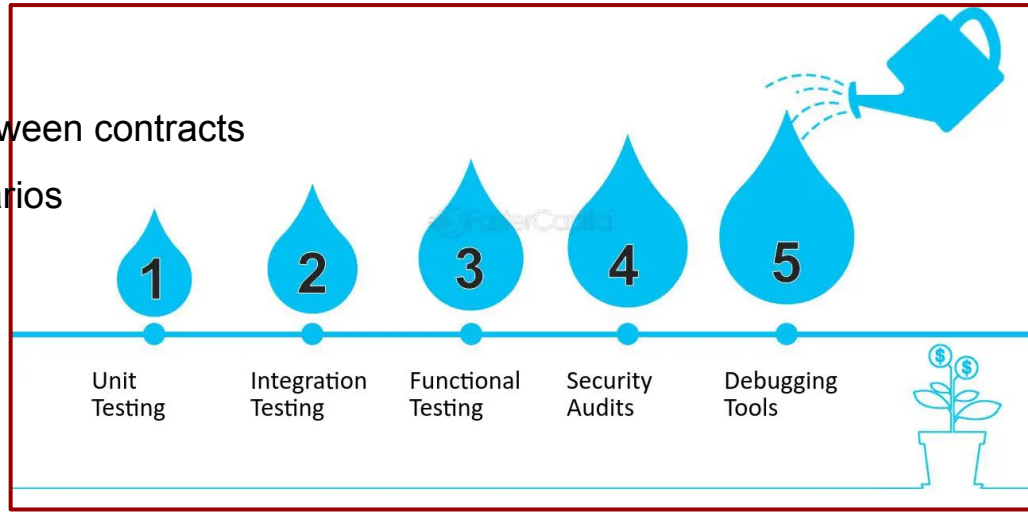
1. Using Remix Debugger
   - Step-by-step execution
   - Inspect variables and stack
   - Analyze failed transactions
2. Events for Debugging
   - Emit events to track execution flow
   - Example : `emit Debug("Value reached", value);`
3. Require & Assert
   - require() → for input validation
   - assert() → for internal checks
   - Example : `require(balance > 0, "Insufficient balance");`



Courtesy : Faster Capital

7. Debugging, Testing, and Security

- Prevents costly bugs
- Ensures contract behaves as expected
- Types of Testing
  - **Unit Testing** : Test individual functions
  - **Integration Testing** : Test interaction between contracts
  - **Edge Case Testing** : Test extreme scenarios
- Tools for Testing
  - Hardhat
  - Truffle Suite
  - Foundry



7. Debugging, Testing, and Security

Common Vulnerabilities in smart Contracts

1. Reentrancy Attack
   - External call before state update
   - Prevention:
     - Checks-Effects-Interactions pattern
     - Use **ReentrancyGuard**
2. Integer Overflow/Underflow : Fixed in Solidity ≥0.8 (auto-checks)
3. Denial of Service (DoS) : Caused by loops or failing transactions
4. Front-running : Attackers manipulate transaction order
5. Access Control Issues : Missing or incorrect modifiers

Courtesy : Immune Bytes

8. Best Practices for Smart Contracts

1. Design & Architecture Best Practices

- Keep Contracts Simple
  - Avoid unnecessary complexity
  - Smaller contracts are easier to audit and secure
- Modular Design
  - Split logic into multiple contracts/libraries
  - Use inheritance carefully
- Upgradeability Considerations
  - Use proxy patterns if upgrades are needed
  - Clearly separate storage and logic

8. Best Practices for Smart Contracts

2. Security Best Practices

- Follow Checks–Effects–Interactions Pattern

```
// Checks
        require(balance >= amount);
// Effects
        balance -= amount;
// Interactions
        payable(msg.sender).transfer(amount);
```

- Prevent Reentrancy Attacks
  - Update state before external calls
  - Use guards

    (e.g., OpenZeppelin ReentrancyGuard)

- Use Access Control Mechanisms
  - Restrict sensitive functions using modifiers

```
modifier onlyOwner {
  require(msg.sender == owner);
  _;
}
```

- Validate Inputs
  - Always validate function parameters

```
require(_amount > 0, "Invalid amount");
```

- Avoid Hardcoding Sensitive Data
  - Do not store private keys or secrets on-chain

8. Best Practices for Smart Contracts

3. Gas Optimization Practices

- Minimize storage usage
- Use **memory** instead of **storage** when possible
- Avoid unbounded loops
- Pack variables efficiently

4. Error Handling

- Use require, assert, revert Properly
  - require() → input validation
  - assert() → internal invariants
  - revert() → custom error handling
- Use Custom Errors (Solidity ≥0.8.4)

```
error NotAuthorized();
```

  - Saves gas compared to string messages

5. Testing Best Practices

- Write Comprehensive Tests
  - Unit tests
  - Integration tests
  - Edge case testing
- Use Development Tools
  - Hardhat
  - Foundry
  - Truffle Suite
- Aim for High Coverage
  - Cover all branches and conditions

8. Best Practices for Smart Contracts

6. Use Standard Libraries

- Prefer Audited Libraries
  - Use OpenZeppelin
- Benefits:
  - Pre-audited code
  - Secure implementations (ERC20, Ownable, etc.)

7. Event Logging

- Use Events for Important Actions

```
event Transfer(address indexed from, address indexed to, uint value);
```

- Helps:
  - Debugging
  - Frontend integration
  - Transparency

8. Deployment Best Practices

- Test on Testnets First
  - Use networks like Sepolia or Goerli
- Verify Contract Code
  - Publish source code for transparency
- Use Proper Gas Settings
  - Avoid underpricing or overpaying

9. Upgrade & Maintenance

- Use proxy patterns for upgradeability
- Maintain backward compatibility
- Document changes clearly

8. Best Practices for Smart Contracts

10. Documentation & Readability

- Write Clean Code
  - Meaningful variable names
  - Proper comments
- Maintain Documentation
  - Function descriptions
  - Architecture diagrams

11. Common Pitfalls to Avoid

- Unchecked external calls
- Infinite loops
- Improper access control
- Timestamp dependence
- Ignoring fallback/receive functions

12. Audit & Review

- Perform Security Audits
  - Internal + external audits
- Use Static Analysis Tools
  - Slither, MythX

9. Integration of Smart Contracts with Blockchain Networks

- connecting your **on-chain logic** (written in Solidity) with **blockchain networks, user interfaces, and backend systems** to build complete decentralized applications (DApps) on platforms like Ethereum.

- **Typical DApp architecture**:

  1. Smart Contract Layer (On-chain)
     - Contains business logic
     - Deployed on Ethereum or other networks
  2. Application Layer (Frontend)
     - Web/mobile UI (React, Angular, etc.)
     - Interacts with contracts via Web3 libraries
  3. Middleware Layer
     - APIs, backend services
     - Indexing services, oracles

9. Integration of Smart Contracts with Blockchain Networks

Step 1 : Smart Contract Deployment

- Tools for Deployment
  - Remix IDE (beginner-friendly)
  - Hardhat (advanced workflows)
  - Truffle Suite
- Deployment Steps
  1. Write contract in Solidity
  2. Compile contract
  3. Deploy to:
     - Local network (Ganache)
     - Testnet (Sepolia)
     - Mainnet
  4. Get:
     - Contract Address
     - ABI (Application Binary Interface)

9. Integration of Smart Contracts with Blockchain Networks

Step - 2 : Connecting Frontend to Smart Contracts

- Web3 Libraries
  - Web3.js
  - Ethers.js
- Wallet Integration
  - Use MetaMask
  - Enables:
    - Account access
    - Transaction signing
    - Network switching

- Example (Ethers.js)

```
import { ethers } from "ethers";
const provider = new ethers.providers.Web3Provider(window.ethereum);
const signer = provider.getSigner();
const contract = new ethers.Contract(contractAddress, abi, signer);
await contract.set(10);
```

9. Integration of Smart Contracts with Blockchain Networks

Step - 3 : Interaction with Smart Contracts

Type	Description	Cost
Read (view/pure)	Fetch data	Free
Write (transaction)	Modify state	Gas fee

```
const value = await contract.get(); // read
await contract.set(100);            // write
```

Courtesy : 57blocks.com

9. Integration of Smart Contracts with Blockchain Networks

Step - 4 : Backend Integration

- Why Backend is Needed?
  - Handle complex logic
  - Store off-chain data
  - Improve performance
- Common Backend Tools
  - Node.js + Express
  - Databases (MongoDB, PostgreSQL)
- Use Cases
  - User authentication
  - Data caching
  - Analytics

9. Integration of Smart Contracts with Blockchain Networks

Step - 5 : Oracles (External Data Integration)

- Problem : Smart contracts cannot access external data directly
- Solution: Oracles (Eg. Chainlink)
- Use Cases
  - Price feeds
  - Weather data
  - Sports results

Courtesy : 57blocks.com

9. Integration of Smart Contracts with Blockchain Networks

Step - 6 : Event Handling & UI Updates

- Using Events
- Smart contracts emit events → frontend listens

```
contract.on("DataStored", (user, value) => {
 console.log(user, value);
});
```

- Enables real-time updates in DApps

9. Integration of Smart Contracts with Blockchain Networks

Step - 7 : Layer 2 and Multi-Chain Integration

- Scaling Solutions
  - Polygon
  - Arbitrum
  - Optimism
- Benefits
  - Lower gas fees
  - Faster transactions
  - Better scalability

Courtesy : 57blocks.com

9. Integration of Smart Contracts with Blockchain Networks

Step - 8 : Security Considerations in Integration

- Validate User Inputs (Frontend + Contract)
- Handle Failed Transactions Gracefully
- Avoid Exposing Private Keys
- Use HTTPS & secure APIs

Step - 9 : Testing Integration

- Tools :
  - Hardhat (local testing)
  - Testnets for real-world simulation
- What to Test
  - Contract interaction
  - Wallet connection
  - Error handling

9. Integration of Smart Contracts with Blockchain Networks

Summary : Deployment Workflow (End-to-End)

1. Develop smart contract
2. Test locally
3. Deploy to testnet
4. Build frontend
5. Integrate using Web3/Ethers
6. Connect wallet
7. Deploy full DApp